Day 1 Setup & C Refresh

GCC + VS Code + CLion + Git Repo + README + CMake + QEMU ARM

Embedded Firmware Perspective - বাংলা Practical PDF Guide

Prepared for: Embedded C / Firmware Learning Track

এই ডকুমেন্টের লক্ষ্য: Day 1 শেষে আপনি নিজের machine-এ C code compile/run, Git repo তৈরি, CMake build, এবং QEMU ARM Cortex-M3 firmware demo run করার complete workflow বুঝবেন।

Core output: একটি *clean Git repo + native C Hello + embedded firmware Hello + README + build/run proof.*

সূচিপত্র

1. Day 1 learning outcomes

Day 1-এর উদ্দেশ্য শুধু "Hello World" চালানো না। Firmware engineering-এ কাজ করতে হলে build system, compiler, target architecture, linker, emulator, repo discipline - সবকিছুর basic workflow একসাথে বুঝতে হয়।

- GCC দিয়ে C source file compile, warning-enabled build, এবং executable run করতে পারবেন।
- VS Code ও CLion-এ C/CMake project open, build, run/debug workflow বুঝবেন।
- Git repo initialize, commit, branch এবং remote push করার minimum discipline তৈরি হবে।
- CMake দিয়ে native C project build করতে পারবেন।
- ARM Cortex-M firmware-এর startup, vector table, linker script, semihosting concept বুঝবেন।
- QEMU দিয়ে hardware না থাকলেও basic embedded firmware demo run করার path পাবেন।

Day 1 time-box plan

সময়	Topic	Deliverable
0:00-0:45	Tool install/check	GCC, Git, CMake, Ninja, editor, ARM toolchain, QEMU verify
0:45-1:30	C refresh	main(), printf(), function, / and % operator, pointer-touch
1:30-2:15	GCC CLI	compile flags, warnings, debug symbols
2:15-3:00	VS Code / CLion	open folder, terminal build, CMake integration
3:00-3:45	Git repo + README	clean structure, .gitignore, first commit
3:45-4:45	CMake native build	cmake -S, -B, --build
4:45-6:00	Embedded QEMU demo	ARM toolchain, linker, startup, semihosting output

2. Host C vs Embedded Firmware mental model

Host C মানে আপনার PC/laptop OS-এর উপর চলা program। Embedded firmware মানে microcontroller/SoC target-এর জন্য code, যেখানে OS না-ও থাকতে পারে, এবং memory/register/linker/startup manually control করতে হয়।

Path	Target	Compiler idea	Start point	Output
Host C	PC CPU + OS	gcc main.c -o app	main() থেকে শুরু, OS loader setup করে	printf terminal-এ যায়
Embedded firmware	MCU/SoC target	arm-none-eabi-gcc + linker script	Reset_Handler + vector table থেকে শুরু	UART/ Semihosting/Debug probe লাগে

Build pipeline diagram

```
Source Code (.c)
   ↓ preprocessing
Preprocessed file
   ↓ compile
Assembly
   ↓ assemble
Object file (.o)
   ↓ link with linker script/startup
Executable / Firmware (.exe / .elf)
   ↓ run
PC terminal / QEMU / MCU flash
```

3. Toolchain setup checklist

প্রথমে সব tool install না করে “version check” proof তৈরি করুন। Firmware work-এ ৫০% সমস্যা PATH/toolchain mismatch থেকে আসে।

- [ ] GCC native compiler installed
- [ ] GDB installed for debugging
- [ ] Git installed
- [ ] CMake installed
- [ ] Ninja installed or Make available
- [ ] VS Code installed + C/C++ + CMake Tools extension
- [ ] CLion installed or configured with compiler toolchain
- [ ] Arm GNU Toolchain installed: arm-none-eabi-gcc
- [ ] QEMU installed: qemu-system-arm
- [ ] A clean workspace folder created, e.g., ~/embedded-c/day1

4. Windows / Linux / macOS installation path

Windows recommended path: MSYS2 UCRT64

Windows-এ beginner-friendly ও firmware-friendly setup হলো MSYS2 UCRT64 terminal. এতে GCC, GDB, CMake, Ninja, Git, QEMU package manager দিয়ে install করা যায়।

```
# Open: MSYS2 UCRT64 terminal
pacman -Syu
pacman -S --needed \
  mingw-w64-ucrt-x86_64-gcc \
  mingw-w64-ucrt-x86_64-gdb \
  mingw-w64-ucrt-x86_64-cmake \
  mingw-w64-ucrt-x86_64-ninja \
  mingw-w64-ucrt-x86_64-qemu \
  git
```

PATH note: Windows terminal/VS Code/CLion থেকে command চালাতে চাইলে C:\msys64\ucrt64\bin PATH-এ যুক্ত করতে হবে।

Linux path

```
# Debian/Ubuntu style
sudo apt update
sudo apt install build-essential gdb cmake ninja-build git qemu-system-arm

# ARM cross toolchain package name may vary by distro
sudo apt install gcc-arm-none-eabi
```

macOS path

```
xcode-select --install
brew install gcc cmake ninja git qemu
```

```
# Arm GNU Toolchain: install official Arm package or use a package manager formula if available.
```

5. Verification commands

Install করার পরে এই commands run করে output README.md-তে paste করুন।

```
gcc --version
gdb --version
cmake --version
ninja --version
git --version
qemu-system-arm --version
arm-none-eabi-gcc --version
```

Expected idea: প্রতিটি command version print করবে। “command not found” বা “not recognized” হলে PATH/toolchain install ঠিক হয়নি।

6. Git repo structure

Day 1 repo ছোট কিন্তু professional হওয়া উচিত। Build output source code folder-এর মধ্যে রাখা যাবে না; সবসময় build/ folder ignore করবেন।

```
embedded-c-day1-starter/
├── README.md
├── .gitignore
├── 01-native-hello/
│   ├── CMakeLists.txt
│   ├── include/app.h
│   └── src/main.c
└── 02-qemu-cortexm3/
    ├── CMakeLists.txt
    ├── arm-none-eabi.cmake
    ├── linker.ld
    ├── startup.c
    └── src/main.c
```

Git commands

```
mkdir embedded-c-day1-starter
cd embedded-c-day1-starter
git init
git status
git add .
git commit -m "init: add day1 embedded c starter"

git log --oneline
git branch
```

Remote push করতে হলে GitHub/GitLab-এ empty repository তৈরি করে নিচের command দিন।

```
git remote add origin https://github.com/<your-user>/embedded-c-day1-starter.git
git branch -M main
git push -u origin main
```

7. C refresh: Hello, /, %, function, pointer-touch

Firmware C শেখার জন্য প্রথম দিনেই clean main(), ছোট function, integer division, remainder এবং pointer output parameter বুঝতে হবে। Embedded driver code-এ pointer দিয়ে status/result return করা খুব common।

01-native-hello/src/main.c

```
#include <stdio.h>
#include "app.h"

int add(int a, int b) {
    return a + b;
}

int safe_divide(int a, int b, int *result) {
    if (b == 0 || result == NULL) {
        return -1;
    }

    *result = a / b;
    return 0;
}

int main(void) {
    int a = 20;
    int b = 6;
    int div_result = 0;

    printf("Hello Embedded C Day 1!\n");
    printf("a + b = %d\n", add(a, b));
    printf("a / b = %d, remainder = %d\n", a / b, a % b);

    if (safe_divide(a, b, &div_result) == 0) {
        printf("safe_divide result = %d\n", div_result);
    }

    return 0;
}
```

Symbol	Meaning	Example	Firmware relevance
/	integer division	20 / 6 = 3	দুই operand int হলে decimal অংশ কেটে যায়
%	remainder/modulus	20 % 6 = 2	timer tick, circular buffer index, periodic task-এ useful
&	address-of	&div_result	variable-এর memory address নেয়
*	dereference	*result = value	pointer যেই address দেখাচ্ছে সেখানে value write করে

C refresh rules

- main() সবসময় int main(void) লিখুন যদি command-line argument না লাগে।
- Header file-এ function declaration; .c file-এ implementation রাখুন।

- Compile warning ignore করবেন না। Warning firmware bug-এর early signal।
- Pointer parameter validate করুন: result == NULL হলে write করবেন না।
- Division করার আগে divisor zero কিনা check করুন।

8. GCC compile workflow

GCC দিয়ে সরাসরি compile করলে build system ছাড়া compiler flow বুঝা যায়। CMake শেখার আগে এই command lineটা বুঝতে হবে।

```
cd 01-native-hello

gcc -std=c11 -Wall -Wextra -Wpedantic -g   -Iinclude src/main.c -o day1_hello

./day1_hello
```

Important GCC flags

Flag	Use
-std=c11	C language version set করে
-Wall -Wextra	important warning enable করে
-Wpedantic	standard compliance warning
-g	debug symbols generate করে; GDB/debugger-এ দরকার
-Iinclude	header search path
-o day1_hello	output executable name

9. VS Code setup

VS Code lightweight editor। Embedded C-তে terminal-first workflow রাখুন: build/run command terminal থেকে verify করুন, তারপর extension configure করুন।

- Install extensions: C/C++ by Microsoft, CMake Tools, Cortex-Debug optional, GitLens optional.
- Open the repo folder, single file না। Folder open করলে include path, CMake, Git সব কাজ করবে।
- Terminal select করুন: Windows হলে MSYS2 UCRT64 বা PowerShell with PATH; Linux/macOS হলে default shell।
- Command Palette -> CMake: Configure -> compiler select -> CMake: Build।

Optional VS Code tasks.json for native GCC

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "build-native-gcc",
      "type": "shell",
      "command": "gcc",
      "args": [
        "-std=c11", "-Wall", "-Wextra", "-g",
        "-I01-native-hello/include",
        "01-native-hello/src/main.c",
        "-o", "build/day1_hello"
      ],
      "group": "build",
      "problemMatcher": ["$gcc"]
    }
```

```
  ]
}
```

10. CLion setup

CLion CMake-native IDE। CMakeLists.txt থাকলে CLion project auto-detect করতে পারে। Embedded firmware-এর জন্য আলাদা Toolchain/Profile configure করতে হয়।

Need	Where	Action
Native C	Settings -> Build, Execution, Deployment -> Toolchains	GCC/GDB/CMake/Ninja detect করুন
CMake profile	Settings -> CMake	Build directory: cmake-build-debug বা build/native
Embedded ARM	New CMake profile	CMake options-এ -DCMAKE_TOOLCHAIN_FILE=.../arm-none-eabi.cmake
Run QEMU	External Tool / Terminal	Build শেষে qemu-system-arm command run করুন

CLion-এর সুবিধা: CMake integration strong; অসুবিধা: embedded target run/debug setup beginner-এর জন্য একটু বেশি configuration-heavy।

11. Native CMake project

CMake হলো build system generator। আপনি CMakeLists.txt লিখবেন; CMake আপনার platform অনুযায়ী Ninja/Make/Visual Studio build file generate করবে।

01-native-hello/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)
project(day1_native_hello C)

set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)

add_executable(day1_hello
    src/main.c
)

target_include_directories(day1_hello PRIVATE include)

target_compile_options(day1_hello PRIVATE
    -Wall
    -Wextra
    -Wpedantic
)
```

Build commands

```
cmake -S 01-native-hello -B build/native -G Ninja
cmake --build build/native
./build/native/day1_hello
```

Windows-এ executable সাধারণত .exe হবে: ./build/native/day1_hello.exe

12. Embedded firmware perspective

Firmware-এ শুধু main.c থাকলেই program start হয় না। MCU reset হলে CPU vector table থেকে initial stack pointer ও Reset_Handler নেয়। Linker script memory কোথায় থাকবে তা বলে।

Component	Meaning	Why needed
startup.c	Vector table + Reset_Handler	CPU reset হলে প্রথম code path
linker.ld	Memory map: FLASH/RAM address	firmware কোন address-এ বসবে
arm-none-eabi-gcc	Bare-metal ARM compiler	PC executable না; target ELF বানায়
QEMU	Emulated hardware	physical board ছাড়াই basic run test
Semihosting	Target code থেকে host console I/O	UART না থাকলেও debug print দেখা যায়

Memory-mapped register idea

```
#define GPIOA_ODR_ADDR  (0x40020014UL)
#define GPIOA_ODR       (*(volatile unsigned int *)GPIOA_ODR_ADDR)

void led_on(void) {
    GPIOA_ODR |= (1U << 5);   // register bit set
}
```

volatile ছাড়া compiler ধরে নিতে পারে value change হচ্ছে না, তাই optimization করে repeated read/write বাদ দিতে পারে। Hardware register, ISR-shared flag, DMA-updated buffer-এ volatile গুরুত্বপূর্ণ।

13. QEMU ARM Cortex-M3 project

এই demo-টি lm3s6965evb Cortex-M3 board model দিয়ে semihosting print করে। Hardware board লাগবে না। তবে এটি learning/demo path; real board-এর startup/linker/vendor CMSIS আলাদা হবে।

Toolchain file

02-qemu-cortexm3/arm-none-eabi.cmake

```
set(CMAKE_SYSTEM_NAME Generic)
set(CMAKE_SYSTEM_PROCESSOR ARM)
set(CMAKE_TRY_COMPILE_TARGET_TYPE STATIC_LIBRARY)

set(TOOLCHAIN_PREFIX arm-none-eabi)

set(CMAKE_C_COMPILER ${TOOLCHAIN_PREFIX}-gcc)
set(CMAKE_ASM_COMPILER ${TOOLCHAIN_PREFIX}-gcc)
set(CMAKE_OBJCOPY ${TOOLCHAIN_PREFIX}-objcopy)
set(CMAKE_SIZE ${TOOLCHAIN_PREFIX}-size)
```

CMakeLists.txt

02-qemu-cortexm3/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)
project(qemu_cortexm3_hello C ASM)

add_executable(firmware.elf
    startup.c
    src/main.c
)

target_compile_options(firmware.elf PRIVATE
```

```
    -mcpu=cortex-m3
    -mthumb
    -ffreestanding
    -fdata-sections
    -ffunction-sections
    -Wall
    -Wextra
    -g
)

target_link_options(firmware.elf PRIVATE
    -T${CMAKE_SOURCE_DIR}/linker.ld
    -nostdlib
    -Wl,--gc-sections
    -Wl,-Map=${CMAKE_BINARY_DIR}/firmware.map
)

add_custom_command(TARGET firmware.elf POST_BUILD
    COMMAND ${CMAKE_SIZE} $<TARGET_FILE:firmware.elf>
)
```

linker.ld

02-qemu-cortexm3/linker.ld

```
ENTRY(Reset_Handler)

MEMORY
{
    FLASH (rx)  : ORIGIN = 0x00000000, LENGTH = 256K
    RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 64K
}

_estack = ORIGIN(RAM) + LENGTH(RAM);

SECTIONS
{
    .isr_vector :
    {
        KEEP(*(.isr_vector))
    } > FLASH

    .text :
    {
        *(.text*)
        *(.rodata*)
    } > FLASH

    .data :
    {
        *(.data*)
    } > RAM AT > FLASH

    .bss :
    {
        *(.bss*)
        *(COMMON)
    } > RAM
```

```
}
```

startup.c

02-qemu-cortexm3/startup.c

```
extern int main(void);
extern unsigned long _estack;

void Reset_Handler(void);
void Default_Handler(void);

__attribute__((section(".isr_vector")))
const void *vector_table[] = {
    &_estack,
    Reset_Handler,
    Default_Handler,
    Default_Handler,
    Default_Handler,
    Default_Handler,
    Default_Handler,
    0,
    0,
    0,
    0,
    Default_Handler,
    Default_Handler,
    0,
    Default_Handler,
    Default_Handler
};

void Reset_Handler(void) {
    (void)main();
    while (1) {
        /* Stop here if semihosting exit is not used. */
    }
}

void Default_Handler(void) {
    while (1) {
    }
}
```

main.c with semihosting

02-qemu-cortexm3/src/main.c

```
static void semihost_write0(const char *message) {
    register int operation asm("r0") = 0x04;      /* SYS_WRITE0 */
    register const char *argument asm("r1") = message;

    asm volatile (
        "bkpt 0xAB"
        :
        : "r" (operation), "r" (argument)
        : "memory"
    );
}
```

```
int main(void) {
    semihost_write0("Hello Embedded C from QEMU Cortex-M3!\n");
    semihost_write0("Day 1 firmware run OK.\n");

    while (1) {
        /* Firmware normally never returns. */
    }

    return 0;
}
```

Build and run commands

```
cmake -S 02-qemu-cortexm3 -B build/qemu -G Ninja \
  -DCMAKE_TOOLCHAIN_FILE=$PWD/02-qemu-cortexm3/arm-none-eabi.cmake

cmake --build build/qemu

qemu-system-arm -M lm3s6965evb -cpu cortex-m3 -nographic \
  -semihosting-config enable=on,target=native \
  -kernel build/qemu/firmware.elf
```

Expected output: Hello Embedded C from QEMU Cortex-M3! এবং Day 1 firmware run OK. যদি QEMU exit না করে, Ctrl+A তারপর X চাপুন।

Security note: semihosting host-এর I/O facility ব্যবহার করতে পারে, তাই শুধু trusted code semihosting enabled অবস্থায় চালান।

14. README.md template

একটি ভালো README শুধু project description না; কীভাবে build/run করতে হবে তার exact reproducible instruction।

README.md

```
# Embedded C Day 1 Starter

এই repo-তে Day 1 এর জন্য দুইটি runnable path আছে:

1. `01-native-hello` - PC/host machine-এ normal C program build/run.
2. `02-qemu-cortexm3` - ARM Cortex-M3 bare-metal firmware style project, QEMU + semihosting দিয়ে run.

## Required tools

- Git
- GCC for native build
- CMake
- Ninja optional but recommended
- Arm GNU Toolchain: `arm-none-eabi-gcc`
- QEMU with `qemu-system-arm`

## 1) Native C run

```bash
cmake -S 01-native-hello -B build/native -G Ninja
```
```

```
cmake --build build/native
./build/native/day1_hello
```

Windows PowerShell/MSYS2:

```bash
./build/native/day1_hello.exe
```

2) QEMU Cortex-M3 firmware run

Linux/macOS/MSYS2 terminal:

```bash
cmake -S 02-qemu-cortexm3 -B build/qemu -G Ninja \
  -DCMAKE_TOOLCHAIN_FILE=$PWD/02-qemu-cortexm3/arm-none-eabi.cmake
cmake --build build/qemu
qemu-system-arm -M lm3s6965evb -cpu cortex-m3 -nographic \
  -semihosting-config enable=on,target=native \
  -kernel build/qemu/firmware.elf
```

Expected output:

```text
Hello Embedded C from QEMU Cortex-M3!
Day 1 firmware run OK.
```

If QEMU does not exit automatically, press `Ctrl+A`, then `X`.

Git first commit

```bash
git init
git add .
git commit -m "init: add day1 embedded c starter"
```

15. Troubleshooting

Problem	Likely cause	Fix
gcc: command not found / not recognized	GCC install হয়নি বা PATH missing	gcc --version; Windows হলে C:\msys64\ucrt64\bin PATH add
CMake cannot find compiler	Wrong terminal/toolchain selected	VS Code/CLion terminal থেকে gcc --version verify করুন
ninja not found	Ninja install হয়নি	Ninja install করুন বা -G Ninja বাদ দিয়ে default generator use করুন
arm-none-eabi-gcc not found	ARM toolchain PATH missing	Arm GNU Toolchain install করে bin PATH add করুন
QEMU opens but no text	Semihosting disabled বা wrong machine	-semihosting-config enable=on,target=native এবং -M lm3s6965evb check করুন
undefined reference / link error	linker script/startup missing বা file	CMakeLists source list ও -T linker.ld

	path wrong	verify করুন
VS Code red underline but build works	IntelliSense config mismatch	CMake Tools configure করুন বা compile_commands.json enable করুন
CLion QEMU run not automatic	Run configuration native executable ধরে নিচ্ছে	Terminal/External Tool দিয়ে qemu-system-arm command run করুন

16. Day 1 submission checklist

[ ] README.md contains tool versions

[ ] Git repo has at least 3 commits: setup, native hello, qemu firmware

[ ] Native CMake build successful

[ ] Native app output pasted into README

[ ] QEMU command documented

[ ] QEMU output pasted into README or screenshot saved

[ ] .gitignore excludes build/ and generated binaries

[ ] You can explain host build vs embedded target build in 3-5 sentences

17. Practice questions

কেন `build/` folder Git-এ commit করা উচিত না?

কারণ build output generated artifact। অন্য machine/configuration-এ rebuild করা উচিত। Generated file commit করলে repo dirty, conflict-prone ও non-reproducible হয়।

`gcc` আর `arm-none-eabi-gcc` এর main পার্থক্য কী?

`gcc` host machine-এর জন্য executable বানায়; `arm-none-eabi-gcc` bare-metal ARM target-এর জন্য ELF/object বানায়।

CMake toolchain file কেন দরকার?

Cross compilation-এ build machine আর target machine আলাদা। Toolchain file CMake-কে compiler, target system, processor, objcopy/size ইত্যাদি বলে দেয়।

Firmware-এ linker script কেন গুরুত্বপূর্ণ?

MCU memory map fixed। FLASH/RAM address, vector table placement, stack address linker script ছাড়া ঠিকভাবে control করা যায় না।

Semihosting কী কাজে লাগে?

Target firmware থেকে host console/file I/O ব্যবহার করে debug print দেখা যায়, বিশেষ করে UART driver না থাকলে।

`volatile` কখন দরকার?

Hardware register, ISR-shared flag, DMA-updated memory-এর মতো compiler-এর বাইরে value change হতে পারে এমন memory access-এ।

`/` এবং `%` embedded code-এ কোথায় কাজে লাগে?

`/` scaling/average calculation-এ, `%` circular buffer index, periodic task, timer tick wrap-around-এ কাজে লাগে।

18. Official references

নিচের official documentation/source দেখে command ও concept align করা হয়েছে:

- GCC manual: usual invocation is gcc; cross-compiling may use machine-gcc.
- VS Code C/C++ docs: C/C++ extension provides cross-platform C/C++ support.
- CMake tutorial/docs: CMakeLists.txt, cmake_minimum_required(), project(), add_executable(), cmake --build workflow.
- Git docs: git init creates an empty repository; git add stages changed/new contents.
- Arm GNU Toolchain: open-source suite for C/C++/Assembly programming for Arm architecture.
- QEMU docs: QEMU supports ARM system emulation; semihosting enables target-to-host I/O and should be used only with trusted code.

End of Day 1: goal হলো *tool install* নয়, *reproducible build/run workflow.* আপনি যদি *clean repo* থেকে *clone* করে *native app* এবং *QEMU firmware* আবার চালাতে পারেন, *Day 1 successful*।